

图论

陈仕杰

Shanghai University

May, 2024



前言

讲课内容视时间会有所缩减，详细代码限于排版不作展示，删减内容及代码详见本人博客。

- 最短路问题：
<https://www.cnblogs.com/LittleDrinks/p/17592593.html>
- 并查集：
<https://www.cnblogs.com/LittleDrinks/p/17763322.html>
- 最小生成树：
<https://www.cnblogs.com/LittleDrinks/p/18835144.html>
- tarjan：
<https://www.cnblogs.com/LittleDrinks/p/17623823.html>

- ① 最短路
- ② 最小生成树
- ③ Tarjan

Floyd

Bellman-ford

CF2041D

Dijkstra

gym103409E

gym103409K

gym105386J

③ Tarjan

最短路问题概述

最短路问题 (SP): 求出图上两点之间总权值之和最小的路径。

- 常见算法：
 - Floyd：任意两点间最短路，时间复杂度 $O(n^3)$ 。
 - Bellman-Ford/SPFA：单源最短路，能处理负环，时间复杂度 $O(nm)$ 。
 - Dijkstra：单源最短路，不能处理负环，堆优化后时间复杂度 $O((n + m) \log n)$ 。

松弛操作: 若 $u \rightarrow k \rightarrow v$ 的路径更短, 则用其更新 $u \rightarrow v$ 的距离。

Floyd 算法

Floyd 算法本质是动态规划。

设 $f[k][u][v]$ 表示经过前 k 个点松弛后 $u \rightarrow v$ 的最短路。

初始化:

$$f[0][u][v] = \begin{cases} w_{u,v} & \text{若 } u、v \text{ 之间有边} \\ +\infty & \text{若 } u、v \text{ 之间无边} \end{cases}$$

状态转移方程:

$$f[k][u][v] = \min\{f[k-1][u][v], f[k-1][u][k] + f[k-1][k][v]\}$$

时间复杂度 $O(n^3)$, 使用滚动数组优化空间后空间复杂度 $O(n^2)$ 。

Floyd 算法

传递闭包

B3611 传递闭包

求 n 个点有向图的传递闭包。

一张图的传递闭包定义为一个 $n \times n$ 的矩阵 $B = (b_{ij})_{n \times n}$, 其中

$$b_{ij} = \begin{cases} 1, & i \text{ 可以直接或间接到达 } j \\ 0, & i \text{ 无法直接或间接到达 } j \end{cases}$$

无向图上任意两点可达性可通过并查集维护。

有向图上任意两点可达性可通过 Floyd 求出。

使用邻接矩阵存图, `std::bitset` 可将时间复杂度优化到 $O(\frac{n^3}{w})$ 。

① 最短路

Floyd

Bellman-ford

CF2041D

Dijkstra

gym103409E

gym103409K

gym105386J

② 最小生成树

③ Tarjan

单源最短路——从 BFS 讲起

从某个点出发，尝试更新周围点的最短路，成功则将其加入队列继续更新，这是最朴素的 BFS 算法。

BFS 在单位边权图上效率高，但在任意边权图上可能退化。

需考虑任意边权图上 BFS 复杂度的退化情况或选定队列内点遍历顺序。

① 最短路

Floyd

Bellman-ford

CF2041D

Dijkstra

gym103409E

gym103409K

gym105386J

② 最小生成树

③ Tarjan

CF2041D

给定二维迷宫的起点和终点，不能跨墙走，同一个方向不能连续走三步，问最少几步可以走到终点。

用 (x, y, d, cnt) 表示到达某个节点时的坐标、方向、步数，然后大力 bfs 即可。

AC 代码

SPFA (Bellman-Ford)

Bellman-Ford 算法指出，在一张无负环的任意边权图上，每花费 $O(m)$ 的时间以任意顺序对所有边进行一次松弛，都可以使最短路的长度加一。因此最多 $O(nm)$ 次松弛操作就可以求出所有点的最短路。

spfa 是 Bellman-Ford 算法的队列实现。

很多时候我们并不需要那么多无用的松弛操作。很显然，只有上一次被松弛的结点所连接的边，才有可能引起下一次的松弛操作。那么我们用队列来维护【哪些结点可能会引起松弛操作】，就能只访问必要的边了。

SPFA (Bellman-Ford)

不难发现，其实 spfa 就是 bfs，它和 Bellman-Ford 算法等价，而 Bellman-Ford 证明了其算法的复杂度最劣为 $O(nm)$ ，所以我们就可以继续愉快地 bfs 了。

稀疏图上 spfa 效率较高，但可以被随意构造的菊花图卡成 $O(nm)$ 。

SPFA (Bellman-Ford)

负环

P3385 【模板】负环

给定一个 n 个点的有向图，判断从顶点 1 出发能否到达负环。
负环的定义是：一条边权之和为负数的回路。

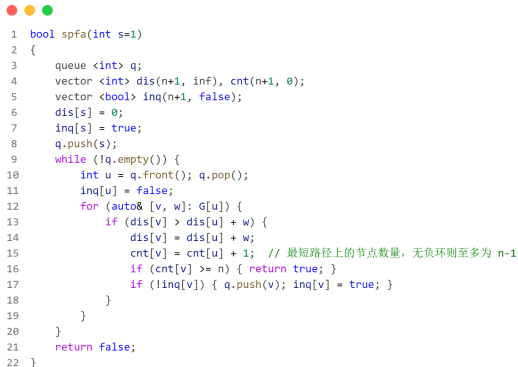
由于负环的存在，边权会越绕越小，因此图中不存在最短路，Dijkstra 会在这里被卡成 TLE。

显然在实际应用中我们无法使用 TLE 判负环法。

使用 spfa，在求最短路的过程中记录一下最短路经过的点的数量 $viscnt$ ，如果 $viscnt$ 大于 $n - 1$ 说明在负环上绕圈了。

SPFA (Bellman-Ford)

负环



```

1  bool spfa(int s=1)
2  {
3      queue<int> q;
4      vector<int> dis(n+1, inf), cnt(n+1, 0);
5      vector<bool> inq(n+1, false);
6      dis[s] = 0;
7      inq[s] = true;
8      q.push(s);
9      while (!q.empty()) {
10         int u = q.front(); q.pop();
11         inq[u] = false;
12         for (auto& [v, w]: G[u]) {
13             if (dis[v] > dis[u] + w) {
14                 dis[v] = dis[u] + w;
15                 cnt[v] = cnt[u] + 1; // 最短路径上的节点数量, 无负环则至多为 n-1
16                 if (cnt[v] >= n) { return true; }
17                 if (!inq[v]) { q.push(v); inq[v] = true; }
18             }
19         }
20     }
21     return false;
22 }

```

图 2: SPFA 判负环代码示例

① 最短路

Floyd

Bellman-ford

CF2041D

Dijkstra

gym103409E

gym103409K

gym105386J

② 最小生成树

③ Tarjan

Dijkstra 算法

Dijkstra 算法指出，在边权非负的图上，尚未访问到的点中，距离最小的那一个最短路一定已经求出。

只需要把 bfs 的队列换成优先队列即可。

模板题 P4779 【模板】单源最短路径（标准版）

Dijkstra 算法

有向图上找最小环

可以使用 Dijkstra 寻找有向图上的最小环。

枚举图上的一个点 u 作为环的起点，跑一遍 Dijkstra 求出点 u 到剩下所有点 v 的最短距离。

如果有一条 $v \rightarrow u$ 的边，那么就找到了一个环，环的权值为 $dis[v] + w(v, u)$ 。

最终时间复杂度 $O(N(N + M) \log M)$ 。

Dijkstra 算法

有向图上找最小环

```

1 void Dijkstra(int s)
2 {
3     priority_queue<pii, vector<pii>, greater<pii>> q;
4     for (int i = 1; i <= n; ++i) { vis[i]=0; dis[i]=inf; }
5     dis[s] = 0;
6     q.push( {0, s} );
7     while (!q.empty()) {
8         int u = q.top().second; q.pop();
9         if (vis[u]) { continue; }
10        vis[u] = 1;
11        for (auto [v,p]: G[u]) {
12            if (dis[v] > dis[u] + p) {
13                dis[v] = dis[u] + p;
14                if (!vis[v]) { q.push( {dis[v], v}); }
15            }
16            // 与 Dijkstra 唯一的区别
17            if (v == s) { mnc = min(mnc, dis[u]+p); }
18        }
19    }
20 }

```

图 3: Dijkstra 求有向图上最小环代码示例

Dijkstra 算法

gym103409E

gym103409E

商店中有 m 条有向边，每条边有一个价格 p_i 。

爱丽丝有 c 块钱，可以从商店中选择若干条边购买，并构成一张图 G 。

鲍勃每次操作可以选择 G 中的一个不带环子图 S 删去。

问鲍勃最少几次操作可以删光 G 中的每一条边。

① 最短路

Floyd

Bellman-ford

CF2041D

Dijkstra

gym103409E

gym103409K

gym105386J

② 最小生成树

③ Tarjan

Dijkstra 算法

gym103409E

一共只有三种情况。

- 如果爱丽丝能够在图上构造出若干个环，那么鲍勃至少需要先用一步把所有环破坏掉，再用一步把剩下的边全部删掉。显然，一种可行的策略是，鲍勃可以选择先删除所有 $u < v$ 的边 $u \rightarrow v$ ，再删除 $u > v$ 的边 $u \rightarrow v$ 。因此如果图上有环，那么鲍勃只需要两步就能结束游戏。
- 如果爱丽丝无法构造出一个环，但有钱买得起若干条边，那么鲍勃只需要一步就可以把边全部删光。
- 如果爱丽丝一条边都买不起，那么鲍勃一步都不需要，游戏就结束了。

于是这道题转化为：求有向图上的最小环。思路见上页。

① 最短路

Floyd

Bellman-ford

CF2041D

Dijkstra

gym103409E

gym103409K

gym105386J

② 最小生成树

③ Tarjan

Dijkstra 算法

gym103409K

gym103409K

n ($n \leq 50$) 座城市, m 条双向道路, 每条路长度为 1。

第 i 条道路由第 c_i 家铁路公司控制。

第 k 次行驶在第 t 家公司行驶的道路上需要花费 $k \times w_t$ 元。

从 1 出发沿着最短路到达其它城市, 若有多条最短路, 选择花费最小的那一条。求最小花费。

Dijkstra 算法

gym103409K

每个点只能由最短路径到达。先跑一遍最短路求出距离，按距离将图分层。然后跑一遍 dfs，当且仅当 $dis[v] = dis[u] + 1$ 时通过 (u, v) 继续向下遍历。

考虑如何计算复杂度。这个问题等价于，将一个正整数 n 划分为若干正整数 a_i 的和，求 $\max\{\prod a_i\}$ 。

首先可以通过平方差公式证明， a 中任意两个数字最多相差 1。然后这个问题变为求 $f(x) = x^{n/x}$ 的最大值。利用对数求导法求得 $f'(x) = \frac{n(1 - \ln x)}{x^2}$ ，当 $x = e$ 时有 $f(x)_{\max} = f(e) = e^{n/e}$ 。

因此，当每层都只有三个点时，有复杂度最大值 $O(3^{\frac{N}{3}})$ ，可以通过本题。

① 最短路

Floyd

Bellman-ford

CF2041D

Dijkstra

gym103409E

gym103409K

gym105386J

② 最小生成树

③ Tarjan

Dijkstra 算法

gym105386J

gym105386J

n 座城市, m 条铁路, 第 i 条铁路由第 c_i 家公司运营, 铁路长度为 l_i 。

有 k 张火车票, 每张只能使用一次。

使用后可以任意乘坐 a_i 公司运营的铁路, 但总距离不能超过 b_i 。

按顺序使用所有车票, 问从 1 能否到达其他城市。

Dijkstra 算法

gym105386J

其实就是最短路，只不过距离需要设计一下。

记现在正在使用第 r 张车票，已经行驶的距离为 d ，定义距离为二元组 (r, d) 。原问题等价于求点 1 到各点的最短路是否存在。

记第 i 张车票为 $tic[i]$ 。设 u 点的最短路为 (r, d) ，经过一条边 (u, v, c, l) 时，如果 $c \neq tic[i].a$ 或者 $d + l > tic[i].b$ ，那么需要换一张车票， v 点距离更新为 (r', l) ；否则继续使用原来的车票， v 点距离更新为 $(r, d + l)$ 。

Dijkstra 算法

gym105386J

考虑如何维护 r' 。 r' 需要同时满足三个条件

$$\begin{cases} r' > r \\ tic[r'].a = c \\ tic[r'].b \geq l \end{cases}$$

记 $rk[a][j] = i$ 表示第 j 张 a 公司的车票为 $tic[i]$, 在 $rk[a]$ 中二分找到第一个 $rk[a][y] > r$ 的 y 。

记 $val[a][j] = b$ 表示第 j 张 a 公司的车票可以行驶 b 的距离, 在 $val[a]$ 中二分找到第一个 $[y, z]$ 最大值大于等于 l 的 z , 则 $r' = rk[a][z]$ 。这一步可以用 ST 表维护。

最终复杂度为 $O(m(\log n + \log k))$ 。

① 最短路

② 最小生成树

并查集

kruskal 算法的证明

gym103729A

瓶颈路

次小生成树

③ Tarjan

① 最短路

② 最小生成树

并查集

kruskal 算法的证明

gym103729A

瓶颈路

次小生成树

③ Tarjan

并查集

将连通块内部视作一棵树，记录每个点的直系父节点。

初始时认为每个点为一个单独的连通块，父节点就是自己。

如果从两个节点向上找到的最远祖先一致，那么它们就在同一个连通块内。

我们只关心某个节点的祖先是谁，而不关心路径上有哪些点。因此在查询到祖先后，可以将自身直接链接到祖先下方，以便后续查询，该优化称为路径压缩。

并查集的时间复杂度是反阿克曼函数，当作 $O(1)$ 即可。

并查集

```
1 struct DSU {
2     int n;
3     vector<int> f;
4     DSU(int n): n(n), f(n+1) {
5         iota(f.begin(), f.end(), 0);
6     }
7     int find(int x) {
8         return f[x]==x? x: f[x]=find(f[x]);
9     }
10    void merge(int x, int y) {
11        x = find(x);
12        y = find(y);
13        if (x == y) { return; }
14        f[x] = y;
15    }
16 };
```

① 最短路

② 最小生成树

并查集

kruskal 算法的证明

gym103729A

瓶颈路

次小生成树

③ Tarjan

kruskal

kruskal 的核心思想是从小到大加入边，如果两条边不在一个连通块内，那么将这条边选入最小生成树，同时合并这两个块。

题目中一般会先求出一张图的最小生成树，然后再在最小生成树上操作，因此实现时我们可以先将所有边存起来，然后通过 kruskal 算法建出最小生成树，并在树上继续进行后续的计算。

kruskal

```
1 vector <array<int,3>> e(m);
2 for (auto& [w, u, v]: e) { cin >> u >> v >> w; }
3 sort(e.begin(), e.end());
4 int ans = 0;
5 DSU d(n);
6 for (auto& [w, u, v]: e) { d.merge(u, v); }
7 for (int i = 1; i <= n; ++i) {
8     if (d.find(i) != d.find(1)) { return -1; }
9 }
10 d = DSU(n);
11 for (auto& [w, u, v]: e) {
12     if (d.find(u) != d.find(v)) {
13         d.merge(u, v);
14         ans += w;
15     }
16 }
```

kruskal 算法的证明

考虑使用反证法，设按照 kruskal 算法得出的生成树为 T ，它和实际的生成树 T' 相差一条边 e 。

从 T 生成 T' 的方式是，在基环树 $T + e$ 的环上找到最小的一条边删去。

由于 kruskal 算法每次选择的都是最小的边，因此满足 $e_T \leq e$ ，因此 T' 必然不优于 T 。

上述证明方法称为“破坏法”，kruskal 算法的证明可以自然地导出【瓶颈路】和【严格次大生成树】的算法。

【瓶颈路】和【严格次大生成树】需要前置知识：倍增 LCA。

① 最短路

② 最小生成树

并查集

kruskal 算法的证明

gym103729A

瓶颈路

次小生成树

③ Tarjan

gym103729A. Nucleic Acid Test

gym103729A. Nucleic Acid Test

给定 n 个点, m 条边的无向图, 其中有 k 个核酸检测点。要求从任意一个核酸检测点出发, 两次核酸之间不能超过 t 分钟, 问若要遍历图上所有点, 并在某个核酸点结束, 你的速度最小是多少。

$1 \leq n \leq 300, 0 \leq t \leq 10^9$, 图不一定连通。

这道题需要综合运用最短路以及最小生成树的知识。

求最小速度可以转化为求最长距离， $v = \left\lceil \frac{l}{t} \right\rceil$ 。

因为我们可以任意选择起点，所以走遍核酸点的最优方案是以每两个核酸点之间的最短路构建出的最小生成树。

然后考虑非核酸点。节点可以重复访问，我们只需要最小化从核酸点走出再走回的时间，因此对某个非核酸点节点来说，最优的方案是从最近的核酸点走出再走回。

由上述分析可知，我们需要走过的最长路径是最小生成树上最长的那条边，以及非核酸点到核酸点最短距离的两倍。根据

$v = \left\lceil \frac{l}{t} \right\rceil$ 计算即可，注意特判 $t = 0$ 的情况。

① 最短路

② 最小生成树

并查集

kruskal 算法的证明

gym103729A

瓶颈路

次小生成树

③ Tarjan

最小瓶颈路

无向图 G 中从 x 到 y 的一条路径，满足路径上最大的边权在所有 x 到 y 的简单路径中最小。

P1967 [NOIP 2013 提高组] 货车运输

A 国有 n 座城市，编号从 1 到 n ，城市之间有 m 条双向道路。每一条道路对车辆都有重量限制，简称限重。

现在有 q 辆货车在运输货物，司机们想知道每辆车在不超过车辆限重的情况下，最多能运多重的货物。

本题是最小瓶颈路模板题。

做法是 `kruskal` 重构树。对图上任意的两点，都可以在重构树上找到一条路径。使用破圈法尝试替换这两点间的路径只会使答案更劣。于是这棵树上两点间的路径即为所求。

为求出这个【最小边权】，只需要修改一下倍增 `LCA` 的算法：在求出 `LCA` 的同时分别求出两点到其 `LCA` 路径上的最小值，再取较小即可。

① 最短路

② 最小生成树

并查集

kruskal 算法的证明

gym103729A

瓶颈路

次小生成树

③ Tarjan

在最小生成树的基础上增加一条边，记其边权为 w ，在形成的环上找到边权小于 w 且最大的那一条边删去。

对于之前没有加入最小生成树中的每一条边，考虑这样操作所形成的所有生成树，其中权值最小的那一棵即为次小生成树。

所谓“环”，实际上就是新加的一条边 $u - v$ ，再加上原本生成树上 u, v 两点间的路径。我们可以通过树上倍增求解次小值。当然由于边权可能相同，如果想求出严格次小生成树，需要同时维护最小值和严格次小值。

模板题：P4180 [BJWC2010] 严格次小生成树

- ① 最短路
- ② 最小生成树
- ③ Tarjan

连通性是图论中的一个重要问题。

对无向图来说，任意两点之间是否连通可以使用并查集维护，并且有一些点或者是边是“关键”的，没有这些点或者边，这个图就会分裂成各个部分。无向图上删边后的连通性（或称双连通性）可以通过分析这些点和边的性质求解。

对有向图来说，任意两点之间是否连通可以使用 Floyd 传递闭包求解。一般的有向图不具有好的性质，而将有向图上可以彼此到达（或称强连通）的若干点缩为一个点，就可以将这张错综复杂的有向图转化为有向无环图，利用 DAG 的有序性进一步求解问题。

tarjan 算法用于求解图的连通性问题。具体而言，它可以求出如下的特殊点、边、连通分量

限制	定义
无向图	桥：删去这条边后，图上原来连通的两点不再连通
无向图	边双连通分量 (EBCC)：任意一条边都不是桥的极大连通子图
无向图	割点：删除该点和所有与其相连的边后，图中原来连通的两点不再连通
无向图	点双连通分量 (PBCC)：任意一个节点都不是割点的极大连通子图
有向图	强连通分量 (SCC)：有向图中任意两点都可互相到达的极大连通子图

记 dfn_u 表示按 dfs 的顺序, u 是第几个遍历到的节点。

记 low_u 表示 u 能到达的点 v 中 dfn_v 的最小值。

dfn_u 和 low_u 都可以在一次 dfs 中通过 dp 求出。而特殊点、边、连通分量都可以通过 dfn 和 low 进行判断。

	判断条件	额外操作
桥	$low[v] > dfn[u]$	无
边双连通分量 (EBCC)	$low[v] > dfn[u]$	忽略所有求完的桥, 进行 dfs, 求得答案
割点	$low[v] \geq dfn[u]$	根节点需要有 2 个及以上满足条件的子节点, 使用 <code>child</code> 变量记录并进行特判
点双连通分量 (PBCC)	$low[v] \geq dfn[u]$	用栈维护子节点, 如果节点 <code>u</code> 通过子节点 <code>v</code> 判断为割点, 那么一路出栈直到 <code>v</code> 出栈, 最后将 <code>u</code> 加入答案中, 但不出栈
强连通分量 (SCC)	$low[u] == low[v]$, 则 <code>u</code> 是这个 SCC 的起始位置	用栈维护子节点, 从栈顶一路删, 直到初始位置出栈

示例代码见jiangly 模板:

<https://www.cnblogs.com/WIDA/p/17633758.html>

Thanks!